Optimize Against EA

Handbook & User Manual

GameLab 2 — Julius-Maximilians-Universität Würzburg

Philipp Sehne
Philipp.Sehne@stud-mail.uni-wuerzburg.de

David Ahrens
David.Ahrens@stud-mail.uni-wuerzburg.de

July 16, 2026

Contents

Chapter 1

Introduction

1.1 What is Optimize Against EA?

Optimize Against EA is an educational web application that teaches optimization concepts — in particular **Evolutionary Algorithms (EAs)** — through a collection of interactive mini-games. Instead of reading about selection, crossover and mutation in the abstract, players *experience* them: they compete against an EA, watch populations evolve step by step, and experiment with the algorithm's parameters to see how behaviour changes.

The application contains three main mini-games, each highlighting a different facet of evolutionary optimization:

Peak Finder (`/PeakFinder`)
: Probe a hidden function surface to find its global minimum — solo, or in competition with an EA whose every generation can be replayed and dissected step by step.

Maze Explorer (`/MazeExplorer`)
: Solve a fogged maze the way an EA would — by hand-writing a fixed-length string of moves — then hand the maze to a real EA, watch its population evolve, and discover how the choice of *fitness function* (Manhattan distance, geodesic distance, length penalty, novelty search) completely changes the algorithm's behaviour.

Shooter Game (`/ShooterGame`)
: Fight waves of enemy agents whose behaviour is encoded as DNA and evolved by a genetic algorithm between rounds — the enemies literally adapt to *your* play style.

1.2 Who is this manual for?

This handbook is aimed at **players and instructors**: it explains how to start the application, how to play each game, and what each control and setting means. No programming knowledge is required.

Developers looking for the programmatic interfaces (engine functions, type contracts, worker protocols) should consult the separate **API documentation** instead.

1.3 Educational goals

Playing through the games, a player encounters the following concepts:

- search spaces, local vs. global optima, deceptive landscapes;
- the evolutionary loop: evaluation → selection → crossover → mutation → new generation;

- elitism and premature convergence;
- fitness shaping — how the *same* EA succeeds or fails depending on how fitness is defined;
- novelty search — why *not* chasing the goal can outperform chasing it;
- continuous genomes (points on a surface), discrete genomes (move sequences), and behavioural genomes (agent DNA).

Chapter 2

Getting Started

2.1 Playing online (recommended)

The easiest way to use the application is the live deployment:

https://OptimizeAgainstEA.web.app

No installation is required. The app runs entirely in the browser (client-side); a modern browser is the only prerequisite.

2.2 System requirements

- **Browser:** a current version of Chrome, Firefox, Edge or Safari.
- **Input:** mouse + keyboard *or* touch screen. No gamepad is required. All games are playable on desktop and on mobile/touch devices (the shooter provides a virtual joystick and aim zone on touch screens); a desktop browser offers the most comfortable experience on very small screens.
- **Network:** only needed to load the page (and for the community Raidboss feature, which uses Firebase).

2.3 Running locally from source

For local use (e.g. offline demos), the app can be built from source. Requirements: Node.js (developed with v24.x; v20+ should work) and `npm`.

1. Open a terminal in `OptimizeAgainstEA/OptimizeAgainstEAApp`.
2. Install dependencies: `npm install`
3. Start the development server: `npm run dev`
   Vite prints a local URL (default `http://localhost:5173`); open it in your browser.

To create a production build instead, run `npm run build`; the static output in `dist/` can be served by any web server (`npm run preview` serves it locally).

2.4 First steps in the app

1. The **home page** (`/`) welcomes you and leads to the **Dashboard** (`/Dashboard`), the central game-selection screen.
2. Pick a game. Each game page has its own mode selector and settings.
3. Look for the **?** help buttons and the **Hints** toggle (light-bulb icon) — see Section 2.5.

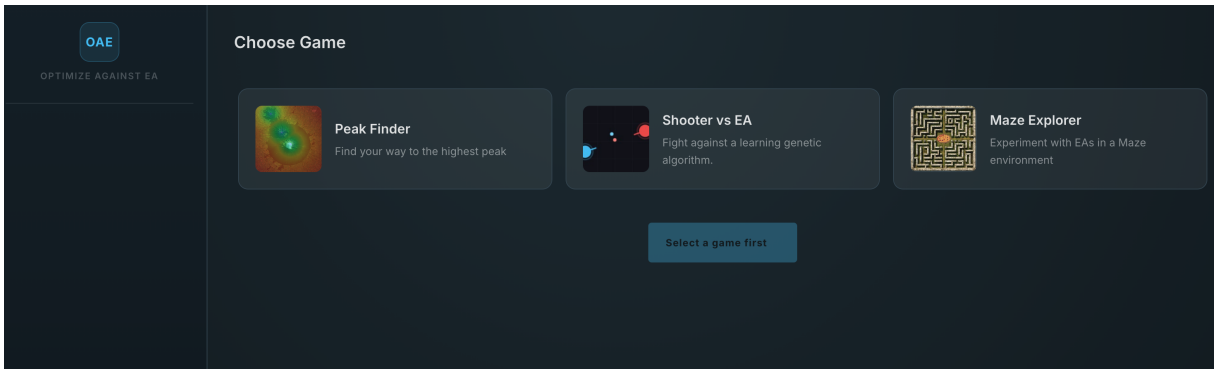

Figure 2.1: The Dashboard — entry point to all mini-games.

2.5 The built-in help and hint systems

The application ships with two layers of in-app assistance:

Help dialogs (? buttons)
Open a modal with two tabs: **Gameplay** (how to play) and **Technical** (what the algorithm is doing). Available on the shooter modes and other game screens.

Contextual hints (light-bulb toggle)
When enabled, short coach-marks and toasts appear at the right moment during play — e.g. after your first probe in Peak Finder, a hint explains that the EA has just advanced a generation. Hints can be switched off at any time and re-armed with the **Reset** button. Your preference is remembered between visits.

Chapter 3

Peak Finder

3.1 Concept

Peak Finder (`/PeakFinder`) is a search-space exploration game in the spirit of *Battleships*. A function surface is hidden from you: every point on the map has a value between 0 (the global minimum — the point you are looking for) and 1 (far away from any minimum). The surface also contains *local* minima — traps that look promising but are not the true goal.

You explore by placing **probes**. Each probe reveals the surface in a small circle around it (as heat map), giving you partial information to plan your next probe. You win by placing a probe inside the **win radius** around the global minimum.

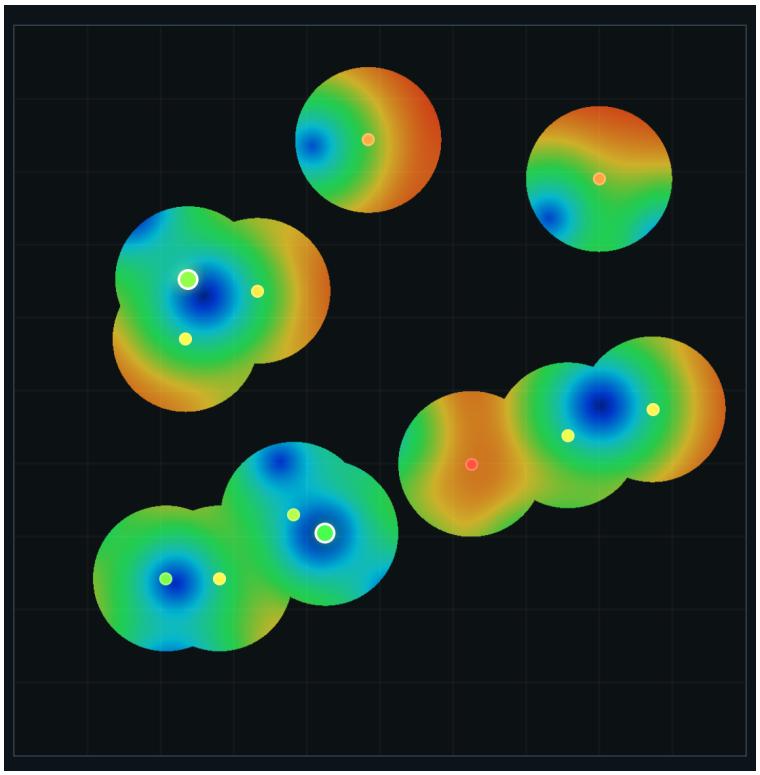

Figure 3.1: Peak Finder: probing a hidden surface. The heatmap is only revealed near your probes — blue basins are promising, the ringed probes mark the best readings so far.

3.2 Game modes

Peak Finder has three modes, selectable at the top of the page:

3.2.1 Play

The classic solo mode. Load a map (a random one, a saved one, or one shared via code — see Section 3.5), then click/tap anywhere to probe. The interface shows your probe history and your best probe so far. Find the global minimum in as few probes as you can.

- **Probe:** click / tap a location on the map.
- **Visualization:** **Heatmap** display of the revealed areas.
- **Win:** a probe inside the win radius triggers the win overlay, which shows your statistics.

3.2.2 Create

Build your own maps. First pick a **map size** (Small, Medium, Large or Huge) — it determines how many minima the map holds (from 4–6 on Small up to 20–26 on Huge), the win radius, and the minimum spacing between minima, which is enforced while you place them. Scatter your local minima, then choose which one is the *global* minimum. Dashed rings preview each minimum's zone. Finished maps can be saved locally and exported as a **map code** to share with others.

3.2.3 Vs EA

The centerpiece mode: you and an Evolutionary Algorithm search the *same* hidden map — and this time it is a race.

- You probe as usual. For every probe you place, the EA advances exactly one **generation** — your move budget and its evolution budget are coupled.
- The EA maintains a **population** of candidate points. It wins when a sufficient fraction of its population (default 35%) gathers inside the win radius.
- Four difficulty presets (**Easy**, **Normal**, **Hard**, **Expert**) bundle an EA configuration with your probe *reveal radius*: harder presets give the EA a bigger, more aggressive search and reveal less of the surface to you.
- After each of your probes you can open the **Replay** viewer to inspect exactly what the EA just did (Section 3.4).
- If the EA solves the map first, you may study its solution and try to beat it in a second attempt.

3.3 The EA settings panel

In Vs EA mode you control the algorithm you are playing against. The settings panel exposes the following parameters. The defaults listed below correspond to the **Normal** difficulty preset, which is active when the mode starts:

Setting	Default	Meaning
Population size	30	Number of candidate points the EA maintains per generation. Larger populations explore more but converge more slowly.
Max. generations	200	Upper limit on generations before the EA gives up.
Crossover rate	0.7	Probability that two parents are recombined (instead of copied).
Mutation rate	0.3	Probability that an offspring is mutated.
Mutation strength	0.25	How far a mutation can move a point across the map.
Mutation decay	0.90	Per-generation factor shrinking the mutation strength — the EA takes big steps early and fine-tunes later.
Win fraction	0.35	Fraction of the population that must sit inside the win radius for the EA to win.
Selection	elitist	How parents are chosen: *tournament*, *roulette* or *elitist*.
Crossover	uniform	How two parents combine: *arithmetic* (blend), *uniform* or *single-point*.
Mutation	gaussian	Shape of the random perturbation: *gaussian*, *uniform* or *cauchy* (occasional long jumps).

Experimenting with these is the point of the game: try a huge mutation strength and watch the population scatter; switch selection to *elitist* and watch it converge prematurely into a local minimum.

3.4 The replay viewer

The replay viewer is a full-screen overlay that replays the EA's last generations **phase by phase**:

initial → *sorted* → *elite* → *breeding* → *mutating* → *new generation* → *win check*

The left side shows the population as dots gliding across the map; the right side lists every individual with its fitness and a role badge (**ELITE**, **PARENT A**, **PARENT B**, **CHILD**, **WIN**). Use the playback controls to step forward/backward. A fitness chart tracks the population's best value across generations.

3.5 Sharing maps: map codes

Every map can be exported as a short text code (via the **Code** dialog) and imported on any other device. Codes encode the minima positions, the global minimum and the win radius. Saved maps and saved function problems are also kept in a local sidebar for quick access.

3.6 Benchmark functions

Besides hand-built and random maps, the loader's **Functions** tab offers a library of fourteen analytic benchmark functions, largely drawn from the well-known BBOB test suite — among them Sphere, Rosenbrock, Ackley, Rastrigin, Schwefel, Eggholder and Gallagher's 101 Peaks. These behave a little differently from maps: a function can have several winning areas — any spot that reaches the optimal value counts as a win, and those spots can form larger regions or appear in more than one place. Each function has its own code, so it can be loaded, saved and shared exactly like a map.

Figure 3.2: The replay viewer (phase 1 of 7): the population as dots on the map, ranked by fitness in the side list — every selection, crossover and mutation is made visible in the following phases.

3.7 Function Tuner (developer tool)

The `/FunctionTuner` page is a developer/instructor companion tool for calibrating how the benchmark functions above are displayed: pick a function, drag a slider to adjust its contrast ("sharpen") exponent, and watch the heatmap and win zone update live; a **Re-roll** button sanity-checks the chosen value under the random rotations players see in real play. Nothing on this page affects a running game — it is not linked from the user interface and only informs the shipped per-function display settings.

Chapter 4

Maze Explorer

4.1 Concept

Maze Explorer (`/MazeExplorer`) is a teaching exhibit built around a simple contrast: first *you* solve a maze the way an EA would — by writing a genome of moves and judging it with a fitness function — then a real EA solves the same kind of maze with a whole population at once, under settings you control.

Mazes are either procedurally generated from a visible **seed** or built by hand in Create mode. Because the seed pins down the exact maze, the same maze can be re-run under different settings — the key to the mode's central experiment. The mode selector offers **Create**, **Solve** and **Control the EA**.

4.2 Solve mode: you are the EA

In **Solve** mode you play the role of a single individual. The maze is hidden under fog-of-war, and you cannot walk it freely — like an EA genome, your solution is a **fixed-length string of moves** that you write with the on-screen D-pad or the arrow keys (↑ ↓ ← →). Only when every slot is filled can you **Submit**: a walker then follows your string through the fog, revealing the corridors it passes. A move into a wall is simply *wasted* — the walker stands still for that step.

Two **filmstrips** share one cursor: the top strip replays your last submitted string (each move tinted by how close it landed to the goal), the bottom strip holds your editable draft. After a run you edit individual moves — a hand-made *mutation* — and resubmit, iterating until the walker reaches the goal. A selectable **fitness function** (Section 4.4) scores every run and repaints the strip instantly, letting you see how the very same run is judged differently.

4.3 Control the EA (experiment mode)

Control the EA is the heart of the game. Here an EA solves the maze while you watch and dissect:

- Each **individual** in the population is a complete *path*: a fixed-length sequence of moves (U/D/L/R). A selectable **wall rule** decides what a blocked move does: *waste* (stand still), *break* (the walk ends at the crash), or *repair* (the gene is fixed to a legal move, which is inherited).
- The maze sits behind a **Breed first generation** overlay while you tune the settings — they shape generation 0. After that, each press of **Evolve** breeds the next generation; settings changed mid-run apply from the next generation and are pinned as markers on the fitness chart, so you can see exactly where a change bent the curve.

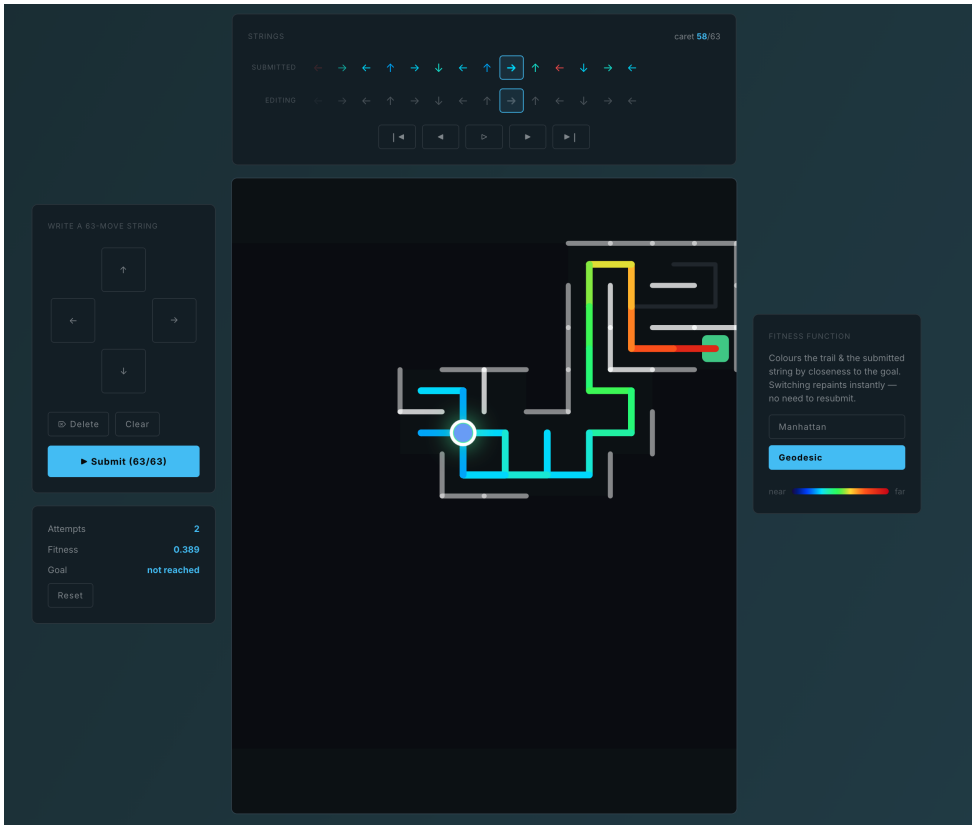

Figure 4.1: Solve mode: a hand-written 63-move string runs under fog-of-war. The filmstrips (top) hold the submitted and the editing string, the D-pad (left) writes moves, and the fitness panel (right) recolours the trail by closeness to the goal.

- The current **best individual** can be played back move by move on the maze, with the rest of the generation walking along as translucent *ghosts* (toggleable): generation 1 is chaotic scribble; over generations the trails braid into a clean route to the goal.
- A per-generation **Dissect last generation** replay (like Peak Finder's, Section 3.4) dissects selection, crossover and mutation. Crossover is especially vivid here: the child follows parent A's route up to the splice cell, then continues on parent B's.

4.4 Fitness functions: the headline experiment

The maze offers several **selectable fitness functions**, and comparing them on the *same seeded maze* is the game's main lesson:

Manhattan (naive objective)
: Rewards ending close to the goal in straight-line (grid) distance, walls be damned. On most mazes this is *deceptive*: it happily rewards a spot that is walled off from the goal, so the population piles up against the wall nearest the goal and gets stuck. Teaches: deceptive landscapes and premature convergence.

Geodesic (BFS field)
: Rewards ending close to the goal *through the corridors* (true walking distance). The same EA now climbs cleanly to the exit. Teaches: fitness shaping can fix a broken search.

Length penalty
: Additionally rewards reaching the goal in fewer moves. Teaches: multi-objective tension between exploration and efficiency.

Novelty search
: Ignores the goal entirely and rewards reaching cells that no previous individual has reached. Counter-intuitively, this often finds the exit when goal-chasing fails — the classic Lehman & Stanley maze result. Teaches: sometimes *not* aiming at the objective works better.

Suggested experiment: note the maze seed, run the EA with Manhattan fitness and watch it get trapped; then re-run the identical maze with geodesic fitness, then with novelty search. Same maze, same algorithm — three completely different behaviours. In Solve mode the same selector exists, so you can also feel the difference on your *own* runs — switching repaints the filmstrip without resubmitting.

4.5 Create mode

Create mode is a maze editor: draw and erase walls on the grid, place the start and the goal, and save the result under a name. Saved mazes are kept locally and can be shared via codes; from the editor (or the setup screen) a maze is handed straight to Solve or Control-the-EA mode, and an **Edit this maze** button in experiment mode brings the current maze back into the editor.

Chapter 5

Shooter Game

5.1 Concept

The Shooter Game (`/ShooterGame`) is a top-down arena shooter with a twist: the enemies are controlled by a **genetic algorithm**. Each enemy agent's behaviour is encoded as a strand of **DNA** — eight genes controlling aggression, dodging, accuracy, preferred range, speed, aim prediction, fire rate and bullet speed. Between rounds the population *evolves*, using a recording ("ghost") of your previous round as training data. The enemies do not just get harder — they adapt to *how you play*.

5.2 Controls

Action	Input
Move	W A S D or arrow keys (desktop) / virtual joystick (touch)
Aim	mouse (desktop) / aim zone (touch)
Shoot	left mouse button or Space (desktop) / aim zone (touch)

There are no further bindings — no dash or similar special moves.

5.3 Game modes

The shooter lobby (`/lobby/shooter`) leads to three modes, each with its own **?** help dialog in-game:

5.3.1 Solo

Round-based duels against evolved agents: land more hits than you take within the 20-second round to win it. After every round the population evolves against your ghost recording; the on-screen **DNA display** shows the current opponent's genes, so you can literally watch traits like *aggression* or *accuracy* rise in response to your tactics. Difficulty presets decide the opponents' starting DNA and how many generations they secretly pre-train before round one. The `/Analytics` page keeps per-round records of your Solo runs.

5.3.2 Horde

A survival mode on obstacle maps — and a showcase of a **steady-state** EA: there are no rounds or waves. Every time you kill an agent, its replacement is bred on the spot from the survivors, so the swarm adapts *continuously* for as long as you stay alive. Agents here carry extra genes (a cyclic movement pattern, body size and opacity — smaller, fainter agents are genuinely harder

to hit, creating real selection pressure). Obstacles matter: solid-bordered ones block bullets — duck behind them for cover — while dashed ones only block movement. During a run you collect stackable **mods** (power-ups) that modify your stats or your shot pattern. Maps are selectable, and a companion **map editor** (`/HordeMapEditor`) lets you build and play your own.

5.3.3 Community Raidboss

A shared, community-evolved opponent: one population of boss DNA lives in the cloud (Firebase). Starting a raidboss session claims one individual from that population; your round against it scores its fitness, and the result flows back into the shared pool, which evolves from everyone's fights. Every player therefore trains the same adversary — the lobby shows how many contributors have shaped it so far.

5.4 Reading the DNA display

Each agent's genome is a vector of eight genes, all in $[0, 1]$:

Gene	**Effect**
Aggression	How strongly the agent chases you.
Dodge weight	How hard it tries to avoid your bullets.
Shoot accuracy	Aim precision (1 = perfect aim).
Preferred range	The combat distance it tries to maintain.
Movement speed	Speed bonus on top of the base speed.
Predict lead	How far it leads its shots ahead of your movement.
Fire rate	How quickly it shoots.
Bullet speed	How fast its bullets travel.

(Horde agents extend this genome with a cyclic movement pattern, body size and opacity — see the Horde mode above.)

Watching these values drift over rounds is the game's evolutionary storytelling: camp in a corner and *accuracy* and *range* genes win; rush constantly and *dodge weight* rises.

5.5 How the evolution works (in brief)

After each round every agent receives a fitness score based on its performance against you (damage dealt, survival, ...). The algorithm then applies tournament selection, single-point crossover and per-gene mutation to produce the next generation. Before your first round, the population can be pre-trained by simulating agents against each other, so even round 1 opponents are not purely random.

Chapter 6

A Small EA Glossary

Plain-language definitions of the terms used across all three games.

Individual
One candidate solution. A point on the map (Peak Finder), a move sequence (Maze), or a DNA vector (Shooter).

Population
The set of individuals alive in one generation.

Fitness
A number scoring how good an individual is. In Peak Finder lower is better (distance-like); in the Shooter higher is better.

Generation
One full cycle of evaluate → select → breed → mutate.

Selection
Choosing which individuals get to reproduce. *Tournament*: pick a few at random, the best of them wins. *Roulette*: chance proportional to fitness. *Elitist*: only the best reproduce.

Crossover
Combining two parents into a child. *Arithmetic*: blend their values. *Single-point*: take the first part from one parent, the rest from the other. *Uniform*: coin-flip per gene.

Mutation
Randomly perturbing a child. Keeps the population exploring instead of collapsing onto one solution.

Elitism
Copying the best few individuals unchanged into the next generation so progress is never lost.

Local optimum
A solution better than all its neighbours but worse than the global best — where greedy search gets stuck.

Deceptive landscape
A problem where following the apparent gradient leads *away* from the true optimum (see the Euclidean maze fitness, Section 4.4).

Premature convergence
The whole population collapsing onto one (often local) optimum before exploring enough.

Novelty search

Selecting for *new behaviour* instead of fitness — rewarding individuals that reach places no one has reached before.

Chapter 7

Troubleshooting & FAQ

The page loads but a game screen stays blank.
Make sure your browser is up to date (the app uses modern Web Worker and Canvas APIs). Try a hard reload (Ctrl+Shift+R / Cmd+Shift+R).

The EA in Vs EA mode never seems to move.
The EA only advances when *you* probe — it runs a fixed number of generations per probe. Place a probe, then open the replay viewer.

Hints stopped appearing.
First check that the light-bulb toggle is on — it is remembered between visits. Peak Finder and Maze Explorer hints re-appear whenever their trigger occurs; only a few one-time hints (e.g. the shooter lobby tour invitations) are shown once per session — the **Reset** button next to the hint toggle re-arms those, and the lobby tours can always be re-run via **Take the Tour**.

My saved maps / mazes disappeared.
Saved content is stored in your browser's local storage. It does not transfer between browsers or devices, and clearing site data deletes it. Use map codes (Section 3.5) to back up or transfer maps.

Performance is poor on my machine.
Reduce the EA population size in the settings panel, and in the maze experiment turn off the ghost walkers. The EAs themselves run in background Web Workers, so the UI should stay responsive even with heavy settings.

Does the app collect data?
The app runs client-side and requires no account. Only the community Raidboss mode communicates with a server (Firebase): it stores the shared boss population — DNA vectors with their fitness statistics and a contributor counter. No personal data is transmitted.

Appendix A

Route Reference

Route	Page
`/`	Home / welcome page
`/Dashboard`	Game selection dashboard
`/PeakFinder`	Peak Finder (Chapter 3)
`/MazeExplorer`	Maze Explorer (Chapter 4)
`/ShooterGame`	Shooter Game (Chapter 5)
`/lobby/shooter`	Shooter lobby / mode selection
`/HordeGame`	Horde mode
`/HordeMapEditor`	Horde map editor
`/FunctionTuner`	Function display tuning tool (developer tool, not linked from the UI)
`/Analytics`	Per-round analytics of Solo shooter runs